HEISTED
Security Review



JULY, 2026

www.chaindefenders.xyz
https://x.com/DefendersAudits

Lead Auditors



PeterSR



0x539.eth

Table of Contents

1. Protocol Summary
2. Disclaimer
3. Risk Classification
4. Audit Details

- Commit Hash
- Scope
- Roles

5. Executive Summary

- Issues Found
- Issues Statuses
- Security Grade

6. Findings

- High
- Medium
- Low
- Informational
- Gas

Protocol Summary

Heisted is a Solana-based on-chain game program built with Anchor. It implements a PvP bank-raiding economy where players join the game, purchase a Bank, recruit Crew with varying Grow Power (GP), farm CASH token rewards, open VRF-powered recruit packs, and raid other players' Banks to steal unclaimed rewards. The protocol features a Token-2022 CASH mint with transfer

fees, Ed25519 signature authority gating for core actions (join, raid target permits), MagicBlock VRF integration for pack-opening and raid randomness, an emission schedule with halving intervals over a 52-day season, Bank upgrade levels (Lv1-10) with increasing capacity and daily limits, a role-based Crew system (12 roles, 8 public + 4 hidden), referral rewards (L1/L2), configurable raid success curves based on attacker/defender GP ratio, per-Bank daily pack and raid limits, in-flight raid lock/latch design (one unresolved raid per target + one outgoing per attacker), admin pause/shutdown controls, and 31 instructions spanning admin configuration, Bank management, Crew deployment, pack opening, raid initiation/settlement, and receipt cleanup.

Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

Risk Classification

Impact/Likelihood	High	Medium	Low
High	High	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Audit Details

Commit Hash

899bf519d75afea95b40014edbed30555ae576da

Scope

Scope

Id	Files in scope
1	programs/heisted/src/lib.rs
2	programs/heisted/src/constants.rs
3	programs/heisted/src/errors.rs
4	programs/heisted/src/events.rs
5	programs/heisted/src/state.rs
6	programs/heisted/src/instructions/mod.rs
7	programs/heisted/src/instructions/admin/mod.rs
8	programs/heisted/src/instructions/admin/config.rs
9	programs/heisted/src/instructions/admin/fee_vault.rs
10	programs/heisted/src/instructions/admin/initialize.rs
11	programs/heisted/src/instructions/admin/shutdown.rs
12	programs/heisted/src/instructions/game/mod.rs
13	programs/heisted/src/instructions/game/crew.rs
14	programs/heisted/src/instructions/game/join.rs
15	programs/heisted/src/instructions/game/pack.rs
16	programs/heisted/src/instructions/game/raid.rs
17	programs/heisted/src/instructions/game/receipt.rs
18	programs/heisted/src/instructions/game/rewards.rs
19	programs/heisted/src/instructions/game/upgrade.rs
20	programs/heisted/src/utils/mod.rs

Id	Files in scope
21	programs/heisted/src/utils/account.rs
22	programs/heisted/src/utils/auth.rs
23	programs/heisted/src/utils/emission.rs
24	programs/heisted/src/utils/fee_vault.rs
25	programs/heisted/src/utils/invite.rs
26	programs/heisted/src/utils/level.rs
27	programs/heisted/src/utils/math.rs
28	programs/heisted/src/utils/pack_role.rs
29	programs/heisted/src/utils/pack_rules.rs
30	programs/heisted/src/utils/pause.rs
31	programs/heisted/src/utils/pda.rs
32	programs/heisted/src/utils/signature.rs
33	programs/heisted/src/utils/terminal.rs
34	programs/heisted/src/utils/time.rs
35	programs/heisted/src/utils/token.rs
36	programs/heisted/src/utils/vrf.rs

Roles

Id	Roles
1	Admin
2	Signature Authority
3	Treasury
4	VRF Provider
5	Player

Executive Summary

Issues Found

Severity	Count	Description
High	3	Critical vulnerabilities
Medium	3	Significant risks
Low	7	Minor issues with low impact
Informational	17	Best practices or suggestions
Gas	3	Optimization opportunities

Issues Statuses

Id	Title	Status
H-01	Reward Remainder Permanently Destroyed When Total Active GP Drops To Zero	Fixed
H-02	Defender Can Drain Pending Rewards During Active Raid, Burning Attacker's CASH	Acknowledged
H-03	'shutdown_close_raid_attempt' Does Not Clear Bank Latches, Permanently Blocking Attacker Claims During the Grace Window	Fixed
M-01	Multiple Live Config Reads At Settlement Allow Admin To Retroactively Burn Attacker CASH On In-Flight Raids	Fixed
M-02	Admin Can Change Protocol Parameters Mid-Season With Immediate Effect, Breaking In-Flight Operations and Player Expectations	Fixed
M-03	Reducing 'max_unlocked_bank_level' Can Permanently Lock In-flight Pack Attempts	Fixed
L-01	Hard Shutdown Close Instructions Return Rent To Admin Instead Of Account Owners	Acknowledged
L-02	Join Timestamps Not Validated Against 'season_end_at'	Fixed
L-03	No Constraints On Admin Signer — Any User Can Initialize A Game Config	Acknowledged

Id	Title	Status
L-04	‘JoinPermit‘ Invite Code Is Not Validated On-chain	Acknowledged
L-05	‘shutdown_close_bank‘,‘shutdown_close_crew_inventory‘ And ‘shutdown_close_pack_attempt‘ Leave Orphaned State On Other Accounts	Acknowledged
L-06	Missing VRF Staleness Check in Callback Handlers	Acknowledged
L-07	‘set_emission_params‘ Does Not Validate ‘season_end_at > now‘	Fixed
I-01	Redundant ‘config_seed‘ Constraints Across 32 Instruction Account Structs	Acknowledged
I-02	No On-Chain Post-Raid Cooldown — Same Defender Can Be Raided Repeatedly	Acknowledged
I-03	‘validate_pack_tier‘ Called Redundantly At Settlement Against Immutable Snapshotted Values	Acknowledged
I-04	Config Update Group Constants Defined Inline Rather Than In ‘constants.rs‘	Fixed
I-05	‘claim_grace_ends_at‘ And ‘reclaim_after‘ Set To Identical Timestamps	Fixed
I-06	‘DuplicateCrewAccount‘ Error For Out Of Sync Seqs	Fixed
I-07	Shutdown Reclaim Error Conflation	Fixed
I-08	Shutdown Constants Can Be Moved To The Dedicated File	Fixed
I-09	‘CrewStatus::Dismissed‘ Variant Is Dead Code	Fixed
I-10	Remaining Accounts Not Checked For Writable Flag In Crew And Pack Instructions	Fixed
I-11	‘shutdown_close_crew‘ And ‘shutdown_close_invite_code‘ Do Not Validate Own State Before Close	Acknowledged
I-12	‘read_crew_account‘ Returns ‘CrewOwnerMismatch‘ For Five Unrelated Failures	Fixed
I-13	‘remove_active_crew‘ Returns ‘CrewStatusMismatch‘ When ‘local_seq‘ Not In Active List	Fixed
I-14	‘initiate_raid‘ Returns ‘RaidMinTakeChanged‘ For Three Unrelated Precondition Failures	Fixed

Id	Title	Status
I-15	Unconstrained ‘close_authority‘ Signer on Receipt Accounts	Acknowledged
I-16	Bit 5 Reserved but Unused in Pause Mask	Acknowledged
I-17	‘compute_global_accumulator‘ Sets ‘last_reward_time = now‘ Past ‘season_end_at‘	Acknowledged
G-01	Redundant ‘find_program_address‘ Calls In Handler	Fixed
G-02	Raid Latch Fields ‘raid_lock_started_at‘, ‘raid_lock_status‘, ‘outgoing_raid_started_at‘ Are Redundant	Acknowledged
G-03	Unnecessary ‘mut‘ On Signer Accounts Across Multiple Instructions	Fixed

Security Grade



Based on the audit findings and code quality, the overall protocol security grade is 79.50 / 100.00. This reflects a good security posture with small room for improvement.

Findings

High

High 01 Reward Remainder Permanently Destroyed When Total Active GP Drops To Zero

Location

emission.rs

Description

When `global_stats.total_active_gp == 0`, `compute_global_accumulator` returns early with `reward_remainder: 0`:

```
if global_stats.total_active_gp == 0 {
    return Ok(GlobalAccumulatorView {
        acc_cash_per_gp: global_stats.acc_cash_per_gp,
        reward_remainder: 0,              // <- silently discards prior remainder
        last_reward_time: now,
    });
}
```

The prior `global_stats.reward_remainder` — which accumulated fractional emission units that could not be evenly divided in previous cycles — is silently set to zero and permanently lost. `last_reward_time` is also advanced to `now`, meaning the emission window during the zero-GP period is also consumed without producing any rewards. Both the accumulated remainder and the zero-GP interval emission are destroyed in a single checkpoint call.

This can be triggered by any sequence that brings `total_active_gp` to zero: all players dismiss or deactivate their Crew, or the last player's bank is closed. Any subsequent `drip_rewards`, `batch_update_crew`, or `claim_farming_rewards` call will hit this branch and commit the destruction.

Proof of Concept

Let's have the following situation:

```
global_stats.reward_remainder = 500    <- accumulated over many cycles
global_stats.total_active_gp  = 1
global_stats.last_reward_time = T=100

Player deactivates last Crew -> total_active_gp = 0
  (checkpoint runs during deactivation, remainder correctly carried)

T=200: drip_rewards called
  total_active_gp = 0 -> early return
  reward_remainder written as 0       <- 500 units permanently destroyed
  last_reward_time written as 200     <- T=100 to T=200 window also lost

T=300: new player joins, activates Crew -> total_active_gp > 0
  Emission resumes from T=200, not T=100.
  The 500-unit remainder is gone forever.
```

Recommendation

Preserve the existing remainder in the zero-GP branch instead of zeroing it:

```
if global_stats.total_active_gp == 0 {
    return Ok(GlobalAccumulatorView {
        acc_cash_per_gp: global_stats.acc_cash_per_gp,
        reward_remainder: global_stats.reward_remainder, // <- preserve, don't zero
        last_reward_time: now,
    });
}
```

This ensures accumulated fractional emission is not lost when the protocol temporarily has no active stakers. The remainder will be distributed correctly when new GP becomes active and the next checkpoint runs. Note that if Bug 1 is also fixed (remainder dropped entirely), this fix becomes unnecessary — but if the remainder carry-forward is retained, this line must be corrected.

Status

New

High 02 Defender Can Drain Pending Rewards During Active Raid, Burning Attacker's CASH

Location

rewards.rs

Description

`claim_farming_rewards_handler` checks the attacker latch (`outgoing_raid_attempt`) but not the defender latch (`in_flight_raid_attempt`):

```
require!(
    ctx.accounts.bank.outgoing_raid_attempt.is_none(),
    HeistedError::RaidOutgoingLatchActive
);
// in_flight_raid_attempt is never checked
```

`Bank` has two distinct raid latches:

- `outgoing_raid_attempt` — set on the attacker's bank when they initiate a raid
- `in_flight_raid_attempt` — set on the defender's bank when they are being raided

When `initiate_raid` runs, `in_flight_raid_attempt` is set on the defender's bank and the defender's `pending_farming_rewards` is snapshotted as `snapshot_pending`. The attacker's CASH cost is burned at this point.

Because `claim_farming_rewards` only checks the attacker latch, a defender with an active raid against them can call `claim_farming_rewards` at any time — including between `initiate_raid` and `settle_raid`. This zeros `pending_farming_rewards`. When `settle_raid` subsequently runs, it reads the live `pending_farming_rewards` (now 0), computes `current_take = bps_mul_floor(0, raid_take_bps) = 0`, and triggers `MinTakeFailed`. The raid resolves as a failure. The attacker loses their burned CASH cost and receives nothing.

The same gap exists in `claim_non_farming_reward_bucket` at line 398, which also only checks `outgoing_raid_attempt`.

This is exploitable by any defender at any time with no prerequisites — it requires no admin action, no special role, and no timing beyond submitting the claim before the VRF settles.

Proof of Concept

Let's have the following situation:

1. Defender has pending_farming_rewards = 500 CASH
2. Attacker calls initiate_raid:

   -> attacker.outgoing_raid_attempt = Some(raid_pda)

   -> defender.in_flight_raid_attempt = Some(raid_pda)

   -> attacker burns attack_cost CASH (e.g. 20 CASH)

   -> snapshot_pending = 500 stored in RaidAttempt

   -> VRF requested
3. Defender calls claim_farming_rewards:

   -> outgoing_raid_attempt.is_none() = true (defender has no outgoing raid)

   -> in_flight_raid_attempt NOT checked <- bug

   -> pending_farming_rewards zeroed

   -> 500 CASH minted to defender
4. VRF arrives, settle_raid called:

   -> settle_bank_pending_farming runs -> pending = 0

   -> current_take = bps_mul_floor(0, raid_take_bps) = 0

   -> current_take(0) < effective_min_take -> MinTakeFailed

   -> resolved_take = 0
5. Result:

   Attacker: lost 20 CASH, received 0

   Defender: kept 500 CASH + farming continues

Recommendation

Add a guard for the defender latch in both claim handlers:

```
// rewards.rs - claim_farming_rewards_handler, after line 216
require!(
    ctx.accounts.bank.outgoing_raid_attempt.is_none(),
    HeistedError::RaidOutgoingLatchActive
);
// ADD:
require!(
    ctx.accounts.bank.in_flight_raid_attempt.is_none(),
    HeistedError::RaidTargetLocked
);
```

Apply the same fix to `claim_non_farming_reward_bucket` at line 398. This ensures defenders cannot drain their pending rewards while a raid is in flight, preserving the economic guarantee that initiated raids have a valid capture target.

Status

Acknowledged

High 03 `shutdown_close_raid_attempt` Does Not Clear Bank Latches, Permanently Blocking Attacker Claims During the Grace Window

Location

shutdown.rs (handler)

shutdown.rs (accounts struct)

Description

When `enter_hard_shutdown` is called, the protocol opens a 7-day grace window (`claim_grace_ends_at = now + 7 days`) during which players are expected to claim their pending rewards before account reclamation begins.

After `reclaim_after` passes, admin calls `shutdown_close_raid_attempt` to force-terminate in-flight raids and reclaim rent. However, `shutdown_close_raid_attempt_handler` only terminates the `RaidAttempt` account state — it never clears the latch fields on the attacker's and target's bank accounts.

In the normal settlement path, both `settle_raid` and `cleanup_raid_attempt` call `clear_raid_latches`, which zeroes:

- `attacker_bank.outgoing_raid_attempt`
- `attacker_bank.outgoing_raid_target_bank`
- `attacker_bank.outgoing_raid_nonce`
- `attacker_bank.outgoing_raid_started_at`
- `target_bank.in_flight_raid_attempt`
- `target_bank.in_flight_raid_nonce`

- `target_bank.raid_lock_started_at`
- `target_bank.raid_lock_status`

The shutdown close path has no equivalent. Furthermore, `ShutdownCloseRaidAttempt` does not include `attacker_bank` or `target_bank` as accounts, making it structurally impossible to clear the latches in the current implementation.

All three claim instructions check `bank.outgoing_raid_attempt.is_none()` before proceeding:

```
// rewards.rs:213-216 (claim_farming_rewards)
// rewards.rs:397-400 (claim_raid_rewards and claim_referral_rewards)
require!(
    bank.outgoing_raid_attempt.is_none(),
    HeistedError::RaidOutgoingLatchActive
);
```

Because the attacker's `outgoing_raid_attempt` latch is never cleared by the shutdown close path, any attacker who has an in-flight raid at the moment of shutdown is permanently blocked from calling `claim_farming_rewards`, `claim_raid_rewards`, and `claim_referral_rewards` for the entire duration of the grace window, and permanently thereafter.

Proof of Concept

Let's have the following situation:

1. Player A initiates a raid against player B -> `attacker_bank.outgoing_raid_attempt = Some(raid_attempt_key)` is set
2. Admin calls `enter_hard_shutdown` before VRF delivers randomness
3. Grace window opens: `claim_grace_ends_at = now + 7 days`, `reclaim_after = now + 7 days`
4. After `reclaim_after` passes, admin calls `shutdown_close_raid_attempt` for the in-flight raid
5. `terminalize_raid_attempt_for_shutdown` sets `raid_attempt.status = ShutdownExpired`, `resolved_take = 0` — but `attacker_bank.outgoing_raid_attempt` remains `Some(raid_attempt_key)`
6. Player A calls `claim_farming_rewards` during what should be the valid grace window -> reverts with `RaidOutgoingLatchActive`
7. Player A calls `claim_raid_rewards` -> same revert
8. Player A calls `claim_referral_rewards` -> same revert
9. Grace window expires — all of A's claimable rewards are permanently lost

Impact

An attacker with an outgoing raid in flight at the time of `enter_hard_shutdown` permanently loses access to all three of their reward buckets:

- `pending_farming_rewards` — accumulated farming CASH earned before shutdown
- `pending_raid_rewards` — loot from previously settled successful raids
- `pending_referral_rewards` — referral cuts credited from referred players

All balances are stranded in the bank account with no path to claim. The 7-day grace window — the only mechanism designed to protect players during shutdown — is entirely bypassed for these users. The attack cost burned at `initiate_raid` time is additionally unrecoverable.

Recommendation

The least invasive fix is to bypass the outgoing latch check in the claim handlers when the protocol is in hard shutdown and the grace window is still open. A user with a stuck in-flight raid should not be penalised by having their claimable rewards locked for the duration of the grace window.

Status

Fixed (partially) - once in shutdown state, rewards can't be claimed anymore.

Medium

Mid 01 Multiple Live Config Reads At Settlement Allow Admin To Retroactively Burn Attacker CASH On In-Flight Raids

Location

raid.rs (initiation — what IS snapshotted)

raid.rs (settlement — live `raid_take_bps` read)

raid.rs:300-301 (settlement — live `raid_min_take_floor_bps` read)

raid.rs:584-592 (cleanup — live `vrf_fulfillment_timeout_seconds` read)

config.rs (`set_raid_scalars_handler`)

Description

`RaidAttempt` snapshots `success_bps` at initiation time, but three other raid parameters are read live from `GameConfig` at settlement or cleanup time. Because the attacker burns their CASH cost at initiation — before the VRF resolves — admin can change any of these parameters between initiation and settlement to force a `MinTakeFailed` outcome, causing the attacker to lose their burned CASH with no loot received.

Parameter 1: `raid_take_bps` (live read at settlement, `raid.rs:496-499`)

```
// initiation: success_bps IS snapshotted ✓
let success_bps = raid_success_bps( ... );  // stored in RaidAttempt

// settlement: raid_take_bps is NOT snapshotted ✗
let current_take = math::bps_mul_floor(
    ctx.accounts.target_bank.pending_farming_rewards,
    ctx.accounts.game_config.raid_take_bps,  // ← live read
)?;
```

Admin lowers `raid_take_bps` -> `current_take` falls below `effective_min_take` -> `MinTakeFailed` -> attacker loses burned CASH.

Parameter 2: `raid_min_take_floor_bps` (live read at settlement, `raid.rs:300-302`)

```
let floor_min_take = math::bps_mul_floor(snapshot_take, config.raid_min_take_floor_bps)
    ?;
let effective_min_take = clamped_min_take.max(floor_min_take);
```

`snapshot_take` is stored in `RaidAttempt` but `raid_min_take_floor_bps` is not. Admin raises `raid_min_take_floor_bps` to 10000 (100%) -> `floor_min_take = snapshot_take` -> `effective_min_take = snapshot_take`. If the defender's pending rewards have decreased by even 1 unit since initiation, `current_take < effective_min_take` -> `MinTakeFailed`.

Parameter 3: `vrf_fulfillment_timeout_seconds` (live read at cleanup, `raid.rs:588`)

```
let (terminal_status, _) = raid_cleanup_terminal_outcome(
    ... ,
    ctx.accounts.game_config.vrf_fulfillment_timeout_seconds,  // ← live read
    now,
    ...
)?;
```

Admin slashes `vrf_fulfillment_timeout_seconds` to its minimum value while raids are in-flight -> all existing `RaidAttempt` accounts whose VRF hasn't returned yet immediately satisfy `now - requested_at ⩾ timeout` -> `cleanup_raid_attempt` can be called -> raid resolved as timeout failure -> attacker's burned CASH lost.

Proof of Concept

`raid_take_bps` path:

1. raid_take_bps = 2000 (20%), raid_min_take_floor_bps = 10000 (100%)
2. Attacker initiates raid: snapshot_pending=1000, snapshot_take=200, attack_cost=40 CASH burned
3. Admin sets raid_take_bps = 100 (1%)
4. settle_raid: current_take = bps_mul_floor(1000, 100) = 10

   MinTakeFailed: 10 < effective_min_take(200) -> resolved_take = 0

   Attacker: lost 40 CASH, received 0

`vrf_fulfillment_timeout_seconds` path:

1. vrf_fulfillment_timeout_seconds = 3600 (1 hour), raid initiated 30 min ago
2. Admin sets vrf_fulfillment_timeout_seconds = 60 (1 minute)
3. now - requested_at = 1800s > 60s -> raid is immediately timeout-eligible
4. cleanup_raid_attempt called -> terminal_status = VrfTimeout

   Attacker: lost attack_cost CASH, received 0

Recommendation

Snapshot all three parameters into `RaidAttempt` at initiation and use the stored values at settlement/cleanup:

```
// Add to RaidAttempt struct:
pub raid_take_bps_snapshot: u16,
pub raid_min_take_floor_bps_snapshot: u16,
pub vrf_fulfillment_timeout_seconds_snapshot: i64,

// At initiate_raid:
attempt.raid_take_bps_snapshot = config.raid_take_bps;
attempt.raid_min_take_floor_bps_snapshot = config.raid_min_take_floor_bps;
attempt.vrf_fulfillment_timeout_seconds_snapshot = config.
    vrf_fulfillment_timeout_seconds;

```

```
// At settle_raid — use snapshots:
let current_take = math::bps_mul_floor(pending, ctx.accounts.raid_attempt.
    raid_take_bps_snapshot)?;
let floor_min_take = math::bps_mul_floor(snapshot_take, ctx.accounts.raid_attempt.
    raid_min_take_floor_bps_snapshot)?;

// At cleanup_raid_attempt — use snapshot:
raid_cleanup_terminal_outcome( ... , ctx.accounts.raid_attempt.
    vrf_fulfillment_timeout_seconds_snapshot, ... )?;
```

This is consistent with how `success_bps` is already snapshotted — the economic parameters of a committed raid should be immutable after the attacker has burned their cost.

Status

Fixed

Mid 02 Admin Can Change Protocol Parameters Mid-Season With Immediate Effect, Breaking In-Flight Operations and Player Expectations

Location

config.rs (`set_emission_params_handler`)

config.rs (`set_pack_and_crew_batch_limits_handler`)

config.rs (`set_raid_scalars_handler`)

config.rs (`validate_emission_params_update`)

Description

Several admin config setters accept parameter changes that take immediate effect with no time-lock, no magnitude constraints, and no protection for operations that were initiated under the old parameters. Three distinct classes of harm arise.

Class A: Emission schedule manipulation (`set_emission_params`)

`set_emission_params_handler` checkpoints `GlobalStats` before applying changes, so past earnings are preserved. However the new parameters govern all future emission instantly with no bounds on how much they can change.

Arbitrary `base_rate` increase — can be set to `MAX_BASE_RATE` in one call, immediately multiplying the per-second emission rate and diluting existing CASH holders with no warning.

Halving interval retroactive repositioning — `halving_index_at` computes the current halving position as an offset from `season_start_at`:

```
let elapsed = timestamp.checked_sub(config.season_start_at)?;
if elapsed < config.first_halving_delay_seconds { return Ok(0); }
let after_first = elapsed - config.first_halving_delay_seconds;
after_first / config.regular_halving_interval_seconds + 1
```

Changing `first_halving_delay_seconds` mid-season moves where the halvings land relative to `season_start_at`. If the season is at month 4 and `first_halving_delay_seconds` is changed from 6 months to 1 month, the formula immediately computes that 3 halvings have already occurred — the live rate drops to `base_rate >> 3` in the same block with no prior warning.

`season_end_at` shortening as covert shutdown — setting `season_end_at = now` immediately halts all future farming emission. `scaled_emission_between` clamps to `season_end_at`, so every subsequent accumulator call returns 0. This achieves a de-facto farming shutdown without going through `enter_hard_shutdown` and its player protections (grace window, latch clearing, claim guarantees).

Class B: `pack_settle_max_chunk_len` reduction breaks pending pack settlements (`set_pack_and_crew_batch_limits`)

`pack_settle_max_chunk_len` is read live at `settle_pack` (pack.rs:478). A player pays CASH at `request_pack` time — the cost is burned before settlement. If admin reduces `pack_settle_max_chunk_len` below the chunk size of a pending request, `settle_pack` reverts. The player's CASH is already spent and the pack is permanently stuck until admin raises the limit back.

Class C: `raid_min_target_pending` raised mid-season (`set_raid_scalars`)

`raid_min_target_pending` is checked live at `initiate_raid` (raid.rs:290-292). Raising it above defenders' current pending balances makes new raids impossible to initiate against small targets, breaking a core game mechanic mid-season without player consent.

Proof of Concept

Class A — halving manipulation:

```
Season: start_at=T0, first_halving_delay=180d, regular_interval=90d, base_rate=1000

T0+120d: admin sets first_halving_delay = 30d
```

```
4
5  halving_index_at(T0+120d):
6    elapsed = 120d ⩾ 30d -> halving active
7    after_first = 90d / 90d + 1 = index 2
8    rate = 1000 >> 2 = 250
9
10 Players expected rate=1000 for 60 more days.
11 Rate drops instantly to 250 — 60d × 750 units/s/GP lost.
```

Class B — pack settlement DoS:

1. pack_settle_max_chunk_len = 10
2. Player calls request_pack with chunk_size=8, CASH burned
3. Admin sets pack_settle_max_chunk_len = 5
4. Player calls settle_pack -> reverts: chunk_size(8) > max(5)
5. Player's CASH is gone, pack is stuck indefinitely

Recommendation

Class A: Prevent `season_end_at` from being shortened and cap `base_rate` change magnitude:

```
require!(next.season_end_at ⩾ current.season_end_at, HeistedError::InvalidParameter);
require!(next.base_rate ⩽ current.base_rate.saturating_mul(2), HeistedError::
    InvalidParameter);
```

For full protection, add a timelock: store `pending_emission_params` + `effective_at = now + MIN_DELAY` and apply in `checkpoint_global_rewards` when `now ⩾ effective_at`.

Class B: Snapshot `pack_settle_max_chunk_len` into `PackAttempt` at request time and use the stored value at settlement.

Class C: Only allow `raid_min_target_pending` increases to take effect for new seasons, not retroactively.

Status

Fixed

Mid 03 Reducing `max_unlocked_bank_level` Can Permanently Lock In-flight Pack Attempts

Location

state.rs (`validate_drop_tables`)

pack.rs (tier re-validation at settlement)

pack_role.rs (tier bound)

Description

If the admin reduces `max_unlocked_bank_level` below the `tier` of an in-flight `PackAttempt`, the pack becomes permanently un-settleable because:

1. `validate_drop_tables` (`state.rs`) validates drop table rows only up to `usize::from(max_unlocked_bank_level)`. Rows above the new max are not validated and may contain all zeros or invalid data if the admin sets them to zero during a config update that also lowers the max level.
2. The settlement chunk handler re-validates the tier but only checks `tier ⩽ pack_attempt.bank_level_at_request` (the frozen request-time bank level). It does not check that the drop table row for that tier is non-zero or valid.
3. `pack_role.rs` (`role_for_roll`) requires the drop table row to sum to `BPS_DENOMINATOR` (10000). A zeroed-out row fails this check.

The net effect: a user with a randomness-ready pack attempt at tier 4 cannot settle it if the admin reduces `max_unlocked_bank_level` below 4 and the tier-4 drop row is no longer valid. The funds (pack cost) are burned at request time and never recoverable.

Proof of Concept

Let's have the following situation:

1. User has Bank at level 5. User calls `request_pack_opening(quantity=1, tier=4)`. Cost is burned. Attempt is awaiting randomness.
2. VRF callback delivers randomness. Attempt is `randomness_ready`.
3. Admin calls a hypothetical `set_max_unlocked_bank_level(3)` (or uses a future `set_drop_tables` that zeros row 3+).
4. Any third party calls `settle_pack_opening_chunk` for this attempt.

5. Settlement calls `pack_role.rs:draw_pack_role` with the attempt's `tier=4` and `bank_level_at_request=5`.
6. `pack_role.rs:17` validates the tier is within `MAX_BANK_LEVEL` (passes — 4 <= 10).
7. `role_for_roll` reads the drop table at index `tier - 1 = 3`. If this row was zeroed by the admin config change, `role_for_roll` reverts with `InvalidParameter` because the row sum != 10000.
8. The attempt is permanently un-settleable. The only remaining action is `cleanup_pack_attempt` (which marks it expired with no refund).

Add the following tests to `programs/heisted/tests/unit/utils/pack_role.rs`:

```
#[test]
fn role_for_roll_rejects_zeroed_row() {
    let zeroed_row = [0u16; ROLE_COUNT];
    let enabled = [false; ROLE_COUNT];
    let result = role_for_roll(&zeroed_row, &enabled, 0);
    assert!(result.is_err());
}

#[test]
fn draw_pack_role_fails_when_tier_drop_row_is_zeroed() {
    let mut drop_tables = DEFAULT_DROP_TABLES;
    drop_tables[4] = [0u16; ROLE_COUNT];
    let enabled = DEFAULT_ROLE_ENABLED;
    let key = Pubkey::new_unique();
    let randomness = [9u8; 32];
    let result = draw_pack_role(&drop_tables, &enabled, 5, &key, &randomness, 0);
    assert!(result.is_err());
}
```

Run with:

```
cargo test -p heisted -- role_for_roll_rejects_zeroed_row
cargo test -p heisted -- draw_pack_role_fails_when_tier_drop_row_is_zeroed
```

Recommendation

Option A (preferred): Enforce that `max_unlocked_bank_level` is monotonic — can only be increased, never decreased. Add this check to any config update that touches it.

Option B: Validate all `MAX_BANK_LEVEL` rows in `validate_drop_tables` rather than only the first `max_unlocked_bank_level` rows. This keeps the full table consistent at all times.

Status

Fixed

Low

Low 01 Hard Shutdown Close Instructions Return Rent To Admin Instead Of Account Owners

Location

shutdown.rs

Description

All six `shutdown_close_*` instructions use Anchor's `close = admin` constraint, which transfers the entire rent-exempt lamport balance of the closed account to the admin wallet:

```
// ShutdownClosePackAttempt  — shutdown.rs:48
#[account(mut, close = admin, seeds = [ ... ], bump, ... )]
pub pack_attempt: Box<Account<'info, PackAttempt>>,

// ShutdownCloseRaidAttempt  — shutdown.rs:77
#[account(mut, close = admin, seeds = [ ... ], bump, ... )]
pub raid_attempt: Box<Account<'info, RaidAttempt>>,

// ShutdownCloseCrew         — shutdown.rs:106
#[account(mut, close = admin, seeds = [ ... ], bump, ... )]
pub crew: Box<Account<'info, Crew>>,

// ShutdownCloseCrewInventory — shutdown.rs:135
#[account(mut, close = admin, seeds = [ ... ], bump, ... )]
pub crew_inventory: Box<Account<'info, CrewInventory>>,

// ShutdownCloseBank         — shutdown.rs:163
#[account(mut, close = admin, seeds = [ ... ], bump, ... )]
pub bank: Box<Account<'info, Bank>>,

// ShutdownCloseInviteCode   — shutdown.rs:191
#[account(mut, close = admin, seeds = [ ... ], bump, ... )]
pub invite_code: Box<Account<'info, InviteCode>>,
```

These accounts were funded by players at their own expense. `Bank`, `CrewInventory`, and `InviteCode` are created during `join_and_create_bank` — the player pays `Rent::minimum_balance(Bank::SIZE)`, `Rent::minimum_balance(CrewInventory::SIZE)`, and `Rent::minimum_balance(InviteCode::SIZE)` from their own wallet (all three `create_reclaimable_account` calls use `payer = &owner` and `refund_to = &owner`). `Crew` accounts are created during pack settlement where the payer is the pack opener. `PackAttempt` and `RaidAttempt` are funded by the player and attacker respectively.

Important clarification on `refund_to`: The `refund_to` field in `create_reclaimable_account` only handles the case where the PDA address was pre-funded before creation — it refunds those pre-funded lamports back to the player at creation time. It has no effect on what happens when the account is later closed via Anchor's `close =` constraint. The `close =` target is the sole determinant of where rent goes on closure, and all six shutdown instructions set it to `admin`.

The non-shutdown close path correctly uses `close = refund_to` pointing to the original owner — visible in `receipt.rs:37` (`ClosePackAttemptReceipt`) and `receipt.rs:95` (`CloseRaidAttemptReceipt`), where `refund_to` is constrained to `pack_attempt.owner` and `raid_attempt.attacker_owner` respectively. The shutdown path breaks this pattern without justification.

There is no player-callable self-close instruction available during shutdown. The 7-day `claim_grace_ends_at` window only allows players to claim pending CASH rewards — it provides no mechanism to recover SOL deposits.

Recommendation

Pass the original account owner as the rent recipient in each shutdown close instruction. Add the owner wallet as an additional account constrained to match the stored owner field:

```
// Example for ShutdownCloseBank:
#[derive(Accounts)]
pub struct ShutdownCloseBank<'info> {
    #[account(mut)]
    pub admin: Signer<'info>,

    pub game_config: Box<Account<'info, GameConfig>>,

    /// CHECK: Original Bank owner — receives rent-exempt lamports on close.
    #[account(
        mut,
        address = bank.owner @ HeistedError::Unauthorized,
    )]
    pub bank_owner: AccountInfo<'info>,

    #[account(
        mut,
        close = bank_owner,  // ← change from close = admin
```

```
        seeds = [BANK_SEED, game_config.config_seed.as_ref(), bank.owner.as_ref()],
        bump,
        ...
    )]
    pub bank: Box<Account<'info, Bank>>,
}
```

Apply the same pattern to the other five instructions, resolving the owner from each account's stored owner field (`pack_attempt.owner`, `raid_attempt.attacker_owner`, `crew.owner`, `crew_inventory.owner`, `invite_code.creator`).

Status

Acknowledged

Low 02 Join Timestamps Not Validated Against `season_end_at`

Location

initialize.rs

Description

In `InitializeSystemParams::validate`, the `InitializeSystemParams` struct carries four time-related fields:

- `allowlist_join_at`
- `public_join_at`
- `season_start_at`
- `season_end_at`

The existing validation enforces:

1. `season_end_at > season_start_at` — the season has a positive duration.
2. `allowlist_join_at ⩽ public_join_at` — the allowlist window opens before or simultaneously with the public window.

What is missing is any check that both join timestamps fall strictly before `season_end_at`. Without this, the join windows can extend past the season boundary. This means a player could submit a join transaction after the season has ended, at which point game logic may be in an unexpected state (e.g., reward accumulation ceased, raids disabled, etc.).

Proof of Concept

Consider the following parameter set that would pass the current validation:

`season_start_at` = T+0

`season_end_at` = T+30 days

`allowlist_join_at` = T+25 days

`public_join_at` = T+35 days

- `require!(season_end_at > season_start_at)` -> ✓ (30 > 0)
- `require!(allowlist_join_at ⩽ public_join_at)` -> ✓ (25 <= 35)
- No check comparing `public_join_at` (T+35) against `season_end_at` (T+30) -> ✗

A user calling the join instruction at T+32 would find the season already ended, potentially causing undefined behavior, transaction reverts that waste user fees, or — in a worst case — a state transition that should not be permitted during an ended season.

Recommendation

Add an additional requirement before or within the existing `require!` block:

```
require!(
    self.public_join_at < self.season_end_at
        && self.allowlist_join_at < self.season_end_at,
    HeistedError::InvalidParameter
);
```

This ensures that both join windows are always fully contained within the `[season_start_at, season_end_at)` interval. Note that `season_start_at` is already implicitly bounded by `allowlist_join_at ⩽ public_join_at` and `season_end_at > season_start_at`, but an explicit lower-bound check (`allowlist_join_at ⩾ season_start_at`) could also be added for defense-in-depth.

Status

Fixed

Low 03 No Constraints On Admin Signer — Any User Can Initialize A Game Config

Location

initialize.rs

Description

The `InitializeSystem` accounts struct declares the admin as an unconstrained `Signer`:

```
#[account(mut)]
pub admin: Signer<'info>,
```

There is no check that this pubkey belongs to a known deployer, a DAO-controlled address, or any authorized entity. Any wallet on Solana can sign this transaction and attempt to initialize a `GameConfig` and `GlobalStats` pair, provided it pays the rent-exempt lamports for the PDA accounts.

Because each game configuration is indexed by a caller-supplied `config_seed: [u8; 32]`, a malicious user cannot overwrite the official config (the PDA derivation is deterministic from the seed), but they can:

1. Deploy a lookalike config with the same `config_seed` prefix.
2. Set malicious parameters such as a different `treasury`, `signature_authority`, or `vrf_oracle_queue`.
3. Front-run the official deployment with a poisoned seed.
4. Socially engineer users into interacting with the rogue config.

Proof of Concept

Let's have the following situation:

1. Alice (malicious) monitors the mempool for the official `InitializeSystem` transaction and extracts the `config_seed`.
2. Alice crafts her own `InitializeSystem` transaction using a different `config_seed` (or even the same seed if the official one hasn't landed — the PDA won't exist yet).
3. Alice sets `treasury` to her own wallet, sets `raid_protected_owner` to a burner, and calls the instruction.
4. Alice's rogue config now exists on-chain.

5. If any front-end or SDK naively indexes all `GameConfig` accounts, or if Alice tricks users via social engineering, players may deposit funds or perform actions against Alice's config.

Recommendation

Several mitigations are possible, and the appropriate choice depends on the deployment model:

Option 1: Hardcode an authorized deployer pubkey

Add a constant in `constants.rs`:

```
pub const AUTHORIZED_DEPLOYER: Pubkey = pubkey!("HeistedDeployerPubkeyHere ... ");
```

Then add a constraint:

```
#[account(mut, constraint = admin.key() == AUTHORIZED_DEPLOYER @ HeistedError::
    Unauthorized)]
pub admin: Signer<'info>,
```

Option 2: Derive config seed from admin pubkey

Require that `config_seed` is a hash of the admin's pubkey (or a known domain), so only the admin can create configs under seeds they control. This doesn't prevent the admin from being malicious, but it binds identity to deployment.

Option 3: Governance gate

Check that `admin` is a PDA owned by a governance program so that initialization requires a DAO vote.

Status

Acknowledged

Low 04 `JoinPermit` Invite Code Is Not Validated On-chain

Location

join.rs

Description

`join_and_create_bank_handler` calls `validate_invite_kind_for_mode` and `validate_permit_referral_shape` to validate the shape of the signed `JoinPermit`, but it never checks whether an on-chain `InviteCode` PDA exists for the `permit.invite_code_hash` or that such an account is `enabled` and non-expired.

The entire invite/admission validity is delegated to the off-chain `signature_authority`. If that key is compromised, or if there is a separation of duties where the `signature_authority` disagrees with the `admin` who disabled a code, an attacker can:

1. Sign a `JoinPermit` referencing a non-existent `invite_code_hash` with arbitrary `recruiter_l1` / `recruiter_l2` pubkeys.
2. Join successfully — the contract creates the Bank, CrewInventory, initial Crew, and a new player `InviteCode`, but never validates the referenced invite code.
3. The emitted `BankCreated` event records the fabricated referral relationship.
4. When the fabricated recruiter Banks later claim, `pending_referral_rewards` is credited to them.

The on-chain `InviteCode` check is redundant because `signature_authority` already controls all permit fields, including `invite_code_hash`. An authority that can sign for a non-existent hash can equally sign for an existent enabled one. The missing check doesn't widen the attack surface — it only misses defense-in-depth if admin and authority are separate roles in the future.

Proof of Concept

Let's have the following situation:

1. Attacker compromises or controls the `signature_authority` key (or the signature authority issues permits for a wallet the attacker controls).
2. Attacker constructs a `JoinPermit` with:
    - `invite_code_hash = hash("NONEXISTENT")` — no on-chain `InviteCode` PDA exists for this hash
    - `recruiter_l1 = attacker_controlled_wallet` — fabricated referral targeting any wallet
3. Sign the permit with the `signature_authority` key.
4. Submit `join_and_create_bank` with the signed permit. The handler:
    - Validates the Ed25519 signature (passes — valid signature)
    - Validates permit shape (passes — `recruiter_l1` is `Some( ... )`)
    - Does not check `invite_code_pda(config_seed, permit.invite_code_hash)` exists, is enabled, or is non-expired

- Creates Bank, CrewInventory, Crew, and player InviteCode
- Writes `bank.recruiter_l1 = fabricated_pubkey`

5. The fabricated recruiter Bank accumulates `pending_referral_rewards` from the new player's farming claims.

Add the following test to `programs/heisted/tests/unit/instructions/game/join.rs`:

```
#[test]
fn permit_accepts_fabricated_invite_code_hash_without_onchain_check() {
    let config = GameConfig {
        player_codes_enabled: true,
        ..base_config(JoinMode::InviteOnly as u8)
    };
    let fabricated_hash = [0xAA; INVITE_CODE_HASH_LEN];
    let fabricated_recruiter = Pubkey::new_unique();
    let permit = JoinPermitInput {
        invite_code_hash: fabricated_hash,
        invite_code_kind: InviteCodeKind::Player as u8,
        recruiter_l1: Some(fabricated_recruiter),
        recruiter_l2: None,
        nonce: 1,
        expires_at: i64::MAX,
    };

    assert!(validate_invite_kind_for_mode(&config, permit.invite_code_kind).is_ok());
    assert!(validate_permit_referral_shape(&permit).is_ok());

    let config_open = GameConfig {
        player_codes_enabled: true,
        ..base_config(JoinMode::Open as u8)
    };
    assert!(validate_invite_kind_for_mode(&config_open, permit.invite_code_kind).is_ok()
    );
}
```

Run with:

```
cargo test -p heisted -- permit_accepts_fabricated_invite_code_hash
```

Recommendation

Add an on-chain check in `resolve_join_permit` that reads the `InviteCode` PDA at `invite_code_pda(config_seed, permit.invite_code_hash)` and verifies:

1. The account exists (deserializes successfully).
2. The `kind` matches the permit's `invite_code_kind`.

3. The `enabled` flag is `true`.
4. The `expires_at` is `None` or in the future.

This requires the `InviteCode` account (or its key) to be included in the `JoinAndCreateBank` account struct or as an optional remaining account. Alternatively, document explicitly that the `signature_authority` is the only gate and accept this as a trust assumption.

Status

Acknowledged

Response: The join instruction does not require the consumed invite/admission code PDA to exist or to be read as an additional account. Backend admission must validate code existence, enabled/disabled state, expiry, wallet binding, use limits, phase rules, and recruiter resolution before issuing the signed permit.

Low 05 `shutdown_close_bank`,`shutdown_close_crew_inventory` And `shutdown_close_pack_attempt` Leave Orphaned State On Other Accounts

Location

shutdown.rs — `shutdown_close_bank_handler`

shutdown.rs — `shutdown_close_crew_inventory_handler`

shutdown.rs — `shutdown_close_pack_attempt_handler`

Description

Three `shutdown_close_*` handlers close PDAs without checking whether other accounts still depend on them. Unlike the self-contained state check issues (missing CrewStatus or invite state checks), these three can orphan or permanently lock state on other accounts.

`shutdown_close_bank_handler` closes a `Bank` without verifying `in_flight_pack_attempt`, `in_flight_raid_attempt`, and `outgoing_raid_attempt` are all `None`. Closing a bank while it has an active raid means the counterparty bank's `in_flight_raid_attempt` or `outgoing_raid_attempt` latch can never be cleared through normal `settle_raid` or `cleanup_raid_attempt` — those instructions attempt to clear both banks' latches, and the closed bank's PDA no longer exists. Additionally, the bank may

hold `pending_farming_rewards`, `pending_raid_rewards`, and `pending_referral_rewards` which are permanently destroyed on close with no disbursement to the owner.

`shutdown_close_crew_inventory_handler` closes a `CrewInventory` without verifying `active_count == 0`. Closing an inventory with active crews orphans those `Crew` accounts — their parent inventory is gone, `role_counts` and `active_crew` state is lost, and they cannot be properly recalled or managed.

`shutdown_close_pack_attempt_handler` closes the `PackAttempt` PDA and returns rent to admin, but the `ShutdownClosePackAttempt` accounts struct does not include the `Bank` account at all — only `admin`, `game_config`, and `pack_attempt`. The handler structurally cannot clear `bank.in_flight_pack_attempt`. After closure the bank latch points to a nonexistent PDA, permanently blocking the player from requesting new pack openings.

```
pub struct ShutdownClosePackAttempt<'info> {
    pub admin: Signer<'info>,
    pub game_config: Box<Account<'info, GameConfig>>,
    pub pack_attempt: Box<Account<'info, PackAttempt>>,
    // MISSING: bank account
}
```

Recommendation

For `shutdown_close_bank_handler`:

```
require!(
    ctx.accounts.bank.in_flight_pack_attempt.is_none()
        && ctx.accounts.bank.in_flight_raid_attempt.is_none()
        && ctx.accounts.bank.outgoing_raid_attempt.is_none()
        && ctx.accounts.bank.pending_farming_rewards == 0
        && ctx.accounts.bank.pending_raid_rewards == 0
        && ctx.accounts.bank.pending_referral_rewards == 0,
    HeistedError::InvalidParameter
);
```

For `shutdown_close_crew_inventory_handler`:

```
require!(
    ctx.accounts.crew_inventory.active_count == 0,
    HeistedError::InvalidParameter
);
```

For `shutdown_close_pack_attempt_handler`: add the `Bank` account to the accounts struct and clear the latch:

```
// Add to context:
pub bank: Box<Account<'info, Bank>>,
// In handler:
ctx.accounts.bank.in_flight_pack_attempt = None;
```

All three enforce the documented close order: terminalize attempts -> recall crews -> close banks/inventories — without relying on admin discipline alone.

Status

Acknowledged

Low 06 Missing VRF Staleness Check in Callback Handlers

Location

pack.rs — `pack_randomness_callback_handler`

raid.rs — `raid_fulfillment_callback_handler`

Description

Both VRF callback handlers accept randomness after the VRF timeout has elapsed. Neither checks whether the callback is stale:

- Pack (`pack_randomness_callback_handler`): Has `expires_at` on `PackAttempt` but doesn't check `now ⩽ expires_at`.
- Raid (`raid_fulfillment_callback_handler`): `RaidAttempt` has no `expires_at` field at all — no check is possible without a struct migration.

```
// pack.rs — expires_at exists but unused
ctx.accounts.pack_attempt.randomness = Some(randomness);
ctx.accounts.pack_attempt.status = PackAttemptStatus::RandomnessReady as u8;

// raid.rs — no expires_at on RaidAttempt
ctx.accounts.raid_attempt.randomness = Some(randomness);
ctx.accounts.raid_attempt.status = RaidAttemptStatus::RandomnessReady as u8;
```

However, settlement still works after a stale callback — neither settlement handler checks `expires_at`. Cleanup is blocked (`RandomnessReady ≠ AwaitingRandomness`), but settlement is the intended path. The stale randomness is still VRF-verified, so the oracle cannot forge the output. This is a defense-in-depth gap with no fund loss or state lock.

Proof of Concept

1. Admin sets `vrf_fulfillment_timeout_seconds = 300`.
2. Player requests a pack/raid. VRF oracle is slow or stale.
3. VRF callback arrives after `expires_at` (pack) or after `requested_at + timeout` (raid).
4. Callback is accepted — no staleness check.
5. Settlement proceeds normally (no `expires_at` check in settlement either).

Recommendation

- Pack (trivial): Add `require!(now ⩽ ctx.accounts.pack_attempt.expires_at, HeistedError::InvalidVrfIdentity)` in `pack_randomness_callback_handler`. The field already exists.
- Raid (requires migration): Add `expires_at` to `RaidAttempt`, populate it at raid initiation, and check it in `raid_fulfillment_callback_handler`. Alternatively, add `game_config` to the callback accounts and compute `requested_at + config.vrf_fulfillment_timeout_seconds`.

Status

Acknowledged

Response: `vrf_fulfillment_timeout_seconds` is a cleanup eligibility timer, not a callback-validity deadline. A valid MagicBlock callback that reaches a still pending attempt before cleanup is accepted even after the timeout.

Low 07 `set_emission_params` Does Not Validate `season_end_at > now`

Location

config.rs — `set_emission_params_handler`

state.rs — `validate_emission_params`

Description

`validate_emission_params` checks `season_end_at > season_start_at` and `season_length ⩽ MAX_SEASON_LENGTH_SECONDS`, but does not require `season_end_at > now`. An admin can accidentally (or intentionally) set `season_end_at` to a past timestamp.

```
pub(crate) fn validate_emission_params(
    base_rate: u64,
    first_halving_delay_seconds: i64,
    regular_halving_interval_seconds: i64,
    season_start_at: i64,
    season_end_at: i64,
) -> Result<()> {
    require!(base_rate > 0 && ...);
    require!(season_end_at > season_start_at, ...);       // OK
    // MISSING: require!(season_end_at > Clock::get()?.unix_timestamp, ...);
    let season_length = season_end_at.checked_sub(season_start_at)?;
    require!(season_length ⩽ MAX_SEASON_LENGTH_SECONDS, ...);
}
```

Critically, `checkpoint_global_rewards` runs before the config update, so accrued rewards up to `now` are checkpointed under the old (correct) `season_end_at`. No user rewards earned before the mistake are lost.

This is Low severity because it is recoverable (the admin can call `set_emission_params` again with a valid `season_end_at > now`), the checkpoint preserves history, and it is admin-triggered only — a fat-finger / misconfiguration scenario, not an external exploit.

Recommendation

Add a runtime guard in the handler (`set_emission_params_handler`, config.rs) after the existing validation call, not in `validate_emission_params` in state.rs. The `validate_emission_params` function is also called during `GameConfig::validate()` in the initialization path, where `now` is obtained later — adding a `now` check would require refactoring the init flow. The simplest targeted fix:

```
validate_emission_params_update(config.paused_functions, current, next)?;
// ADD:
require!(
    params.season_end_at > now,
    HeistedError::InvalidParameter
);
```

Status

Fixed

Informational

Info 01 Redundant `config_seed` Constraints Across 32 Instruction Account Structs

Location

32 occurrences across:

raid.rs (10 occurrences)

pack.rs (5 occurrences)

crew.rs (3 occurrences)

rewards.rs (4 occurrences)

join.rs (1 occurrence)

upgrade.rs (1 occurrence)

config.rs (1 occurrence)

shutdown.rs (6 occurrences)

Description

Every `config_seed == game_config.config_seed` constraint is redundant because the PDA seeds derivation already enforces the same property.

Anchor's `seeds = [BANK_SEED, game_config.config_seed.as_ref(), owner.as_ref()]` constraint works as follows:

1. Takes `game_config.config_seed` from the already-validated `game_config` account
2. Derives the expected PDA address: `find_program_address([BANK_SEED, config_seed, owner], program_id)`
3. Verifies the passed account address equals the derived address
4. If the account was created with a different `config_seed`, it has a different address -> rejected

A Bank account from a different season has a different `config_seed` in its seeds, which produces a different PDA address. It is therefore impossible for it to pass the seeds check. The subsequent `constraint = bank.config_seed == game_config.config_seed` check can never fail independently of the seeds check — if the address passed the seeds derivation, the `config_seed` field inside the account is guaranteed to match.

The same logic applies to every account type that includes `config_seed` in its seeds: `GlobalStats`, `CrewInventory`, `Crew`, `PackAttempt`, `RaidAttempt`, `InviteCode`.

This results in 32 redundant constraint evaluations across the instruction set, adding noise to the codebase and making account structs harder to read without providing any additional security.

Note: `version == ACCOUNT_VERSION` constraints are not redundant — Anchor's `Account<T>` deserialization only checks the 8-byte discriminator, not internal field values. The version check correctly guards against stale accounts after program upgrades.

Similarly, `owner == signer.key()` constraints on accounts whose seeds already include the signer (e.g. `bank.owner == owner.key()` where seeds include `owner.key()`) are technically redundant but are retained intentionally as explicit access control documentation.

Recommendation

Remove all 32 `config_seed == game_config.config_seed` constraints from account structs where `config_seed` is already part of the seeds derivation. The seeds constraint is the authoritative check.

Before removing, verify for each account type that its seeds do include `game_config.config_seed.as_ref()` — the redundancy only holds when `config_seed` appears in the seeds. A representative cleanup for `InitiateRaid`:

```
// BEFORE
#[account(
    mut,
    seeds = [BANK_SEED, game_config.config_seed.as_ref(), attacker_owner.key().as_ref()],
    bump,
    constraint = attacker_bank.version == ACCOUNT_VERSION @ HeistedError::InvalidAccountVersion,
    constraint = attacker_bank.config_seed == game_config.config_seed @ HeistedError::InvalidConfigSeed, // remove
    constraint = attacker_bank.owner == attacker_owner.key() @ HeistedError::Unauthorized,
)]

// AFTER
#[account(
    mut,
    seeds = [BANK_SEED, game_config.config_seed.as_ref(), attacker_owner.key().as_ref()],
    bump,
    constraint = attacker_bank.version == ACCOUNT_VERSION @ HeistedError::
```

```
        InvalidAccountVersion,
17      constraint = attacker_bank.owner == attacker_owner.key() @ HeistedError::
        Unauthorized,
18  )]
```

Status

Acknowledged

Info 02 No On-Chain Post-Raid Cooldown — Same Defender Can Be Raided Repeatedly

Location

raid.rs (`validate_raid_participants`)

shutdown.rs (no post-raid protection setter exists)

Description

After a successful raid, the target bank's `raid_protection_until` field is not updated and no post-raid cooldown is applied. The `clear_raid_latches` function only clears the `in_flight_raid_attempt` latch, which means the same target bank can be raided again the moment the latch is cleared:

```
1  fn clear_raid_latches(attacker: &mut Bank, target: &mut Bank, terminal_status: u8) {
2      target.in_flight_raid_attempt = None;   // latch cleared — immediately raidable
       again
3      target.in_flight_raid_nonce = 0;
4      target.raid_lock_started_at = 0;
5      target.raid_lock_status = terminal_status;
6      // raid_protection_until: never updated
7      ...
8  }
```

Each successful raid only removes `raid_take_bps`% (default 20%) of the defender's `pending_farming_rewards`, not the entire balance:

```
1  // settle_raid:
2  resolved_take = current_take.min(snapshot_take)  // only a % of pending
3  target_bank.pending_farming_rewards -= resolved_take  // 80% remains
```

This means repeated raids against the same target are immediately possible after settlement with 80% of the original pending still available:

```
Raid 1: pending=1000 -> steal 200 -> defender has 800
Raid 2 (immediately): pending=800 -> steal 160 -> defender has 640
Raid 3: pending=640 -> steal 128 -> defender has 512
```

The only on-chain limits on repeated targeting are:

- The attacker's `daily_raid_limits[bank_level]` (total across ALL targets, not per-target)
- The `in_flight_raid_attempt` latch (prevents two simultaneous raids on the same target, but not sequential ones)
- A fresh `RaidTargetPermit` signed by `signature_authority` is required for each raid

The permit requirement is the primary defence against targeted harassment — the off-chain backend controls permit issuance frequency. However this is an off-chain control with no on-chain enforcement.

Recommendation

Document explicitly that post-raid protection is enforced off-chain via permit issuance policy, not on-chain. If stronger on-chain guarantees are desired, consider adding a `last_raided_at` field to `Bank` and a `raid_target_cooldown_seconds` config parameter that prevents re-raiding the same target within a window:

```
// In validate_raid_participants:
require!(
    now ⩾ target.last_raided_at + config.raid_target_cooldown_seconds,
    HeistedError::RaidTargetLocked
);

// In clear_raid_latches (on successful raid):
if resolved_take > 0 {
    target.last_raided_at = now;
}
```

This would make the cooldown transparent and auditable on-chain without relying on off-chain permit policy.

Status

Acknowledged

Info 03 `validate_pack_tier` Called Redundantly At Settlement Against Immutable Snapshotted Values

Location

pack.rs (settlement)

pack.rs (request — where validation is authoritative)

Description

`settle_pack_opening_chunk_handler` calls `validate_pack_tier` at settlement time using values that were written once at request time and never mutated:

```
// Settlement (pack.rs:470-473)
pack_rules::validate_pack_tier(
    ctx.accounts.pack_attempt.tier,
    ctx.accounts.pack_attempt.bank_level_at_request,  // snapshot, never changes
)?;
```

Both `tier` and `bank_level_at_request` are stored in the `PackAttempt` account at request creation (pack.rs:172-173) and have no setter. The same validation already ran at request time:

```
// Request (pack.rs:111)
pack_rules::validate_pack_tier(tier, ctx.accounts.bank.bank_level)?;
```

Since neither input can change between request and settlement, the settlement check is dead validation — it can never fail independently of the request-time check. It adds unnecessary compute and misleads readers into thinking `tier` or `bank_level_at_request` could be externally modified after creation.

Recommendation

Remove the `validate_pack_tier` call at settlement. The request-time check is the authoritative gate.

Status

Acknowledged

Info 04 Config Update Group Constants Defined Inline Rather Than In `constants.rs`

Location

config.rs

Description

The `config.rs` instruction file defines a set of `CONFIG_UPDATE_GROUP_*` constants at its top:

```
const CONFIG_UPDATE_GROUP_PAUSE: u8 = 1;
const CONFIG_UPDATE_GROUP_JOIN_GATE: u8 = 2;
const CONFIG_UPDATE_GROUP_ECONOMY_PRICES: u8 = 3;
const CONFIG_UPDATE_GROUP_PACK_AND_CREW_BATCH_LIMITS: u8 = 4;
const CONFIG_UPDATE_GROUP_RAID_SCALARS: u8 = 5;
const CONFIG_UPDATE_GROUP_RAID_SUCCESS_CURVE: u8 = 6;
const CONFIG_UPDATE_GROUP_EMISSION_PARAMS: u8 = 7;
const CONFIG_UPDATE_GROUP_TREASURY: u8 = 8;
const CONFIG_UPDATE_GROUP_RAID_PROTECTED_OWNER: u8 = 9;
```

These constants carry semantic weight: they are emitted in `ConfigUpdated` events and are consumed by off-chain indexers and backend services to determine which portion of the config changed. As such, they represent a shared contract between on-chain and off-chain code.

The project already has a dedicated `constants.rs` file at `programs/heisted/src/constants.rs` that houses all other system-wide constants (account seeds, sizes, default values, limits, etc.). Placing these group constants there would:

1. Improve discoverability — a developer looking for all configuration-related constants finds them in one place.
2. Reduce collision risk — if another instruction file also defines a local constant with value `8`, there is no cross-file collision check.
3. Centralize documentation — `constants.rs` is the natural place to document the purpose and off-chain contract of each group ID.

Recommendation

Move the constants to `programs/heisted/src/constants.rs` and make them `pub`:

```
pub const CONFIG_UPDATE_GROUP_PAUSE: u8 = 1;
pub const CONFIG_UPDATE_GROUP_JOIN_GATE: u8 = 2;
pub const CONFIG_UPDATE_GROUP_ECONOMY_PRICES: u8 = 3;
pub const CONFIG_UPDATE_GROUP_PACK_AND_CREW_BATCH_LIMITS: u8 = 4;
pub const CONFIG_UPDATE_GROUP_RAID_SCALARS: u8 = 5;
pub const CONFIG_UPDATE_GROUP_RAID_SUCCESS_CURVE: u8 = 6;
pub const CONFIG_UPDATE_GROUP_EMISSION_PARAMS: u8 = 7;
pub const CONFIG_UPDATE_GROUP_TREASURY: u8 = 8;
pub const CONFIG_UPDATE_GROUP_RAID_PROTECTED_OWNER: u8 = 9;
```

Remove the local `const` declarations from `config.rs` — the file already imports `use crate::constants::*;`, so the constants will be available without any additional import changes.

Status

Fixed

Info 05 `claim_grace_ends_at` And `reclaim_after` Set To Identical Timestamps

Location

shutdown.rs (`shutdown_deadlines`)

Description

`shutdown_deadlines` returns the same value for both `claim_grace_ends_at` and `reclaim_after`:

```
fn shutdown_deadlines(now: i64) -> Result<(i64, i64)> {
    let claim_grace_ends_at = now
        .checked_add(SHUTDOWN_CLAIM_GRACE_SECONDS)?;
    Ok((claim_grace_ends_at, claim_grace_ends_at))  // same value for both
}
```

`claim_grace_ends_at` is the last second players can claim. `reclaim_after` is the first second admin can close accounts. The boundary checks use opposite inequalities — claims check `now ⩽ claim_grace_ends_at` and reclaim checks `now > reclaim_after` — so there is no functional overlap at the exact boundary second. However setting both to the same value means the two windows are not explicitly separated, which obscures the intended sequential design and could cause confusion during any future changes to these checks.

Recommendation

To resolve this issue:

```
fn shutdown_deadlines(now: i64) -> Result<(i64, i64)> {
    let claim_grace_ends_at = now
        .checked_add(SHUTDOWN_CLAIM_GRACE_SECONDS)
        .ok_or_else(|| error!(HeistedError::ArithmeticOverflow))?;
    let reclaim_after = claim_grace_ends_at
        .checked_add(1)
        .ok_or_else(|| error!(HeistedError::ArithmeticOverflow))?;
    Ok((claim_grace_ends_at, reclaim_after))
}
```

Status

Fixed

Info 06 `DuplicateCrewAccount` Error For Out Of Sync Seqs

Location

crew.rs

Description

In `validate_sorted_unique_local_seqs`, the `require!` macro uses `HeistedError::DuplicateCrewAccount` when `window[0] ≥ window[1]`, but this covers two distinct cases: (1) a genuine duplicate (`window[0] == window[1]`), and (2) an out-of-order sequence (`window[0] > window[1]`, meaning the input is not sorted ascending). The error variant's name and implied semantics only describe duplication, not ordering violations. A caller receiving `DuplicateCrewAccount` when the actual problem is a misordered list will be misled about what went wrong, making debugging harder.

Recommendation

Introduce a distinct error variant (e.g., `HeistedError::LocalSeqsNotSorted`) and emit it when `window[0] > window[1]`:

```
fn validate_sorted_unique_local_seqs(local_seqs: &[u32]) -> Result<()> {
    for window in local_seqs.windows(2) {
        require!(window[0] ⩽ window[1], HeistedError::LocalSeqsNotSorted);
        require!(window[0] ≠ window[1], HeistedError::DuplicateCrewAccount);
```

```
    }
    Ok(())
}
```

Status

Fixed

Info 07 Shutdown Reclaim Error Conflation

Location

shutdown.rs

Description

In `require_shutdown_reclaim_state`, two `require!` statements use the same `HeistedError::ShutdownRequired` error variant for semantically different conditions:

1. `paused_functions == SHUTDOWN_MODE_MASK` — the system is not in hard shutdown mode
2. `reclaim_after > 0 && now > reclaim_after` — the reclaim window has not yet opened

If the second check fails, the caller receives `ShutdownRequired` even though the system IS in shutdown mode. The error message implies the system needs to enter shutdown first, which is misleading and makes debugging harder. The caller would not know whether the issue is (a) the system isn't in shutdown, or (b) the system is in shutdown but the reclaim window hasn't started.

Recommendation

Introduce a distinct error variant (e.g., `HeistedError::ReclaimWindowNotOpen`) for the timing check:

```
fn require_shutdown_reclaim_state(
    paused_functions: u64,
    reclaim_after: i64,
    now: i64,
) -> Result<()> {
    require!(
        paused_functions == SHUTDOWN_MODE_MASK,
        HeistedError::ShutdownRequired
    );
    require!(
```

```
        reclaim_after > 0 && now > reclaim_after,
        HeistedError::ReclaimWindowNotOpen
    );
    Ok(())
}
```

Status

Fixed

Info 08 Shutdown Constants Can Be Moved To The Dedicated File

Location

shutdown.rs

Description

Seven domain separation tag constants for shutdown summary hashing are defined inline in `shutdown.rs` rather than in `constants.rs`:

```
const SHUTDOWN_PACK_ATTEMPT_SUMMARY_DOMAIN: &[u8] = b"heisted:shutdown-close:pack-attempt:v1";
const SHUTDOWN_RAID_ATTEMPT_SUMMARY_DOMAIN: &[u8] = b"heisted:shutdown-close:raid-attempt:v1";
const SHUTDOWN_CREW_SUMMARY_DOMAIN: &[u8] = b"heisted:shutdown-close:crew:v1";
const SHUTDOWN_CREW_INVENTORY_SUMMARY_DOMAIN: &[u8] = b"heisted:shutdown-close:crew-inventory:v1";
const SHUTDOWN_BANK_SUMMARY_DOMAIN: &[u8] = b"heisted:shutdown-close:bank:v1";
const SHUTDOWN_INVITE_CODE_SUMMARY_DOMAIN: &[u8] = b"heisted:shutdown-close:invite-code:v1";
```

Recommendation

Move these domain tag constants to `constants.rs` alongside other domain separation tags following the project's naming conventions.

Status

Fixed

Info 09 `CrewStatus::Dismissed` Variant Is Dead Code

Location

state.rs

Description

The `CrewStatus` enum includes a `Dismissed = 2` variant, but no instruction in the codebase ever sets a crew's status to `Dismissed`. The only status transitions are:

- `batch_update_crew`: `Idle <-> Active` (recall sets Idle, deploy sets Active)
- `join_and_create_bank`: creates initial crew as `Active`

The `Dismissed` variant appears to be reserved for a future `dismiss_crew` instruction. It is dead code in the current implementation.

Recommendation

Either implement `dismiss_crew` or remove the `Dismissed` variant to reduce confusion. If kept for future use, document the planned semantics.

Status

Fixed

Info 10 Remaining Accounts Not Checked For Writable Flag In Crew And Pack Instructions

Location

crew.rs — `deploy_one_crew`, `recall_one_crew`

pack.rs — `create_idle_crew` (called from `settle_pack_opening_chunk_handler`)

Description

Both `deploy_one_crew` and `recall_one_crew` in `crew.rs` fetch crew accounts from `remaining_accounts` via `find_crew_account`, deserialize them, mutate them, and pass the `AccountInfo` to `pda::write_crew_account` for writing. Similarly, `create_idle_crew` in `pack.rs` fetches crew accounts from

`remaining_accounts` and passes them to `pda::write_crew_account`. In all three cases, the `is_writable` flag on the fetched `AccountInfo` is never checked before writing.

If the runtime did not enforce writability for the account at the expected PDA — for example, if the account was passed at the wrong index or the transaction builder passed a read-only reference — the program will fail with a generic runtime error (`0x0` / `InstructionDidNotExecute`) rather than a descriptive program error. This makes debugging difficult for integrators and obscures the root cause of transaction failures.

Recommendation

After locating each crew account in `remaining_accounts`, verify that `is_writable` is `true` and return a descriptive error if it is not:

```
let crew_info = find_crew_account( ... )?;
require!(crew_info.is_writable, HeistedError::AccountNotWritable);
```

This should be applied consistently in all three locations: `deploy_one_crew`, `recall_one_crew`, and `create_idle_crew`.

Status

Fixed

Info 11 `shutdown_close_crew` And `shutdown_close_invite_code` Do Not Validate Own State Before Close

Location

shutdown.rs

Description

`shutdown_close_crew_handler` and `shutdown_close_invite_code_handler` close PDAs without validating the account's own internal state before closing. Unlike issues involving orphaned state on other accounts, these only affect the account being closed.

`shutdown_close_crew_handler` closes a `Crew` PDA without checking its `CrewStatus`. An `Active` (deployed) crew can be closed directly, even though it contributed `active_gp` to the bank's totals.

If the crew is closed while still `Active` without first being recalled, the bank's `active_gp` field becomes stale — the bank is expected to be closed later, but the on-chain close event alone does not surface whether the crew was active at close time. There is no enforcement of close ordering (recall all crews first, then close individual crew accounts).

`shutdown_close_invite_code_handler` closes an `InviteCode` PDA without checking its `enabled` state, `kind`, or any metadata. An invite code that is still enabled can be closed directly. While all codes are equally defunct during hard shutdown, the close event does not record whether the code was enabled at close time, reducing its utility for off-chain reconciliation.

Both handlers rely entirely on the caller to close in a sensible order, with no on-chain consistency check to catch mistakes.

Recommendation

For `shutdown_close_crew_handler`: Add a check that the crew is not active before closing:

```
require!(
    ctx.accounts.crew.status == CrewStatus::Idle as u8
        || ctx.accounts.crew.status == CrewStatus::Dismissed as u8,
    HeistedError::CrewStatusMismatch
);
```

This enforces that only idle or dismissed crews are closable, preventing accidental closure of active crews before recall. Admin can bulk-recall all crews first, then close individual crew accounts.

For `shutdown_close_invite_code_handler`: Either (a) add a pre-close check that the code is not enabled and emit relevant metadata in the close event, or (b) document explicitly that no pre-close validation is intentional — all codes are equally defunct in hard shutdown and the event is purely informational.

Status

Acknowledged

Response: Hard shutdown reclaim must be able to recover farming/deployed Crew without requiring user recall first, so both idle and active crew remain closeable. The close path now rejects corrupt/unknown status bytes, while final status remains available in the event and summary hash for reconciliation.

Info 12 `read_crew_account` Returns `CrewOwnerMismatch` For Five Unrelated Failures

Location

account.rs — `read_crew_account`

Description

`read_crew_account` validates five distinct properties in a single `require!` joined by `&&`:

```
require!(
    crew.version == ACCOUNT_VERSION
        && crew.config_seed == *config_seed
        && crew.owner == *owner
        && crew.local_seq == local_seq
        && usize::from(crew.role_id) < ROLE_COUNT,
    HeistedError::CrewOwnerMismatch
);
```

The error variant `CrewOwnerMismatch` specifically implies the owner doesn't match, but any of the five conditions could fail: wrong `version`, wrong `config_seed`, wrong `owner`, wrong `local_seq`, or an out-of-range `role_id`. A caller receiving this error cannot determine which check failed without simulating the transaction.

This is the only place `CrewOwnerMismatch` is used, so the issue is not ambiguity with other call sites — it's that one error name covers five unrelated failure modes.

Recommendation

Either split into separate `require!` calls with distinct error variants, or rename to a more generic variant like `InvalidParameter` and log diagnostic data in the event.

Status

Fixed

Info 13 `remove_active_crew` Returns `CrewStatusMismatch` When `local_seq` Not In Active List

Location

crew.rs — `remove_active_crew`

Description

`remove_active_crew` searches for a crew's `local_seq` in the inventory's `active_crew` list. If the sequence is not found, it returns `CrewStatusMismatch`:

```
let position = inventory.active_crew[..count]
    .iter()
    .position(|seq| *seq == local_seq)
    .ok_or_else(|| error!(HeistedError::CrewStatusMismatch))?;
```

The error name `CrewStatusMismatch` suggests the crew's `.status` field (Idle/Active/Dismissed) doesn't match expectations. But the actual failure is that the `local_seq` is not present in the inventory's active list — which could mean the crew was never deployed, was already recalled, the sequence is stale, or the inventory is corrupted. The crew's `status` field is never read in this function.

This error is also used elsewhere for actual status mismatches (e.g., in shutdown close checks), so receiving `CrewStatusMismatch` from `remove_active_crew` could mislead debugging toward checking the crew's status field when the real issue is an inventory inconsistency.

Recommendation

Use a distinct error variant such as `InvalidParameter` or a new `CrewNotInActiveList` variant.

Status

Fixed

Info 14 `initiate_raid` Returns `RaidMinTakeChanged` For Three Unrelated Precondition Failures

Location

raid.rs — `initiate_raid_handler`

Description

Three distinct precondition checks in `initiate_raid_handler` all return the same error variant `RaidMinTakeChanged`:

```
// Check 1: target's pending rewards below minimum threshold
require!(
    snapshot_pending ⩾ config.raid_min_target_pending,
    HeistedError::RaidMinTakeChanged
);

// Check 2: calculated take rounds to zero
let snapshot_take = math::bps_mul_floor(snapshot_pending, config.raid_take_bps)?;
require!(snapshot_take > 0, HeistedError::RaidMinTakeChanged);

// Check 3: attack cost exceeds what the attacker committed
let attack_cost = proportional_cost.max(config.raid_min_cost);
require!(attack_cost ⩽ max_cost, HeistedError::RaidMinTakeChanged);
```

The three failures are semantically unrelated:

1. The defender's pending rewards are too low to raid
2. The calculated take rounds to zero (would be a no-op raid)
3. The attacker didn't authorize enough CASH for the attack cost

The error name `RaidMinTakeChanged` specifically suggests that the minimum take was altered between signing the permit and executing the transaction — which only vaguely relates to check 1. Checks 2 and 3 are entirely different failure modes. A user or integrator receiving this error cannot determine what went wrong without simulating.

Recommendation

Use separate error variants for each check. For example:

- `snapshot_pending < raid_min_target_pending` -> `RaidTargetLocked` or `InvalidParameter`

- `snapshot_take == 0` -> `InvalidParameter` (raid would steal nothing)
- `attack_cost > max_cost` -> existing errors like `InvalidParameter` or a new `RaidCostExceedsMax`

Status

Fixed

Info 15 Unconstrained `close_authority` Signer on Receipt Accounts

Location

receipt.rs — `ClosePackAttemptReceipt`, `CloseRaidAttemptReceipt`

Description

Both `close_pack_attempt_receipt` and `close_raid_attempt_receipt` accept a `close_authority` signer with no constraint on who it must be:

```
pub close_authority: Signer<'info>,
```

Any wallet can invoke the close instruction, though the close is only permitted for receipts that are no longer in-flight — the bank constraint (`bank.in_flight_pack_attempt ≠ Some(pack_attempt.key())`) and the `require_pack_receipt_closeable` / `require_raid_receipt_closeable` checks gate access. Active attempts cannot be closed. The `refund_to` is constrained to the attempt owner, so rent returns to the correct party. Griefing is limited to closing a receipt one second before the owner planned to (no state loss).

Recommendation

If intentional, add a comment documenting the permissionless design. If tighter control is desired, constrain `close_authority` to `bank.owner`.

Status

Acknowledged

Info 16 Bit 5 Reserved but Unused in Pause Mask

Location

constants.rs — pause mask bit constants

Description

The pause mask bit constants skip from bit 4 to bit 6:

```
pub(crate) const PAUSE_JOIN: u64 = 1 << 0;
pub(crate) const PAUSE_CLAIM: u64 = 1 << 1;
pub(crate) const PAUSE_CREW: u64 = 1 << 2;
pub(crate) const PAUSE_PACK_REQUEST: u64 = 1 << 3;
pub(crate) const PAUSE_RAID_INITIATE: u64 = 1 << 4;
pub(crate) const PAUSE_ADMIN_UPDATES: u64 = 1 << 6;
pub(crate) const PAUSE_BANK_UPGRADE: u64 = 1 << 7;
```

Bit 5 (value `1 << 5`) is not defined. This creates a gap in the bit numbering that could cause confusion when reading pause mask values or when adding new pause functions in the future.

Recommendation

Either define a `PAUSE_*` constant for bit 5 or add a comment documenting that bit 5 is reserved for future use.

Status

Acknowledged

Info 17 `compute_global_accumulator` Sets `last_reward_time = now` Past `season_end_at`

Location

emission.rs — `compute_global_accumulator`

Description

When `total_active_gp > 0`, `compute_global_accumulator` sets `last_reward_time = now` after computing the accumulator delta:

```
Ok(GlobalAccumulatorView {
    acc_cash_per_gp,
    reward_remainder,
    last_reward_time: now,  // not min(now, season_end_at)
})
```

Although `scaled_emission_between` correctly bounds the computation to `min(end_time, season_end_at)`, the stored `last_reward_time` is the raw clock timestamp. This causes bookkeeping drift: `last_reward_time` advances past `season_end_at`. If the season is later extended, the next checkpoint computes emission from the drifted value (not from `season_end_at`), so the gap between `season_end_at` and the drifted `last_reward_time` is skipped — no backfill. The field is also misleading for debugging.

Recommendation

Cap `last_reward_time` at `season_end_at` after accumulation:

```
let last_reward_time = now.min(config.season_end_at);
```

Status

Acknowledged

Gas

Gas 01 Redundant `find_program_address` Calls In Handler

Location

initialize.rs

Description

In `initialize_system_handler`, the code re-derives the same PDA addresses that Anchor already validated during account deserialization:

```
let (expected_config, _) =
    Pubkey::find_program_address(&[CONFIG_SEED, config_seed.as_ref()], &crate::id());
let (expected_stats, _) =
    Pubkey::find_program_address(&[GLOBAL_STATS_SEED, config_seed.as_ref()], &crate::id
    ());
```

These PDAs are passed to `pda::create_reclaimable_account()` as the `expected_key` parameter — a safety check that the derived address matches the one being created. However, the same derivation has already occurred inside the Anchor framework to validate the account seeds in the `#[account(seeds = [...], bump)]` attributes.

`Pubkey::find_program_address` is an expensive operation: it iterates bump seeds from `255` downward, calling `sha256` on each attempt until it finds a valid Ed25519 point. This consumes ~200 SHA-256 hashes per call, two calls per initialization — all for a value that could be read directly.

Anchor exposes the already-verified key and bump through:

- `ctx.accounts.game_config.key()`
- `ctx.accounts.global_stats.key()`
- `ctx.bumps.game_config`
- `ctx.bumps.global_stats`

Recommendation

Replace the `find_program_address` calls with direct key access:

```
let expected_config = ctx.accounts.game_config.key();
let expected_stats = ctx.accounts.global_stats.key();
```

The bumps (`config_bump`, `stats_bump`) are already correctly sourced from `ctx.bumps`, so the `signer_seeds` arrays remain correct.

The `expected_key` parameter in `create_reclaimable_account` will still get the correct value — it just won't pay for redundant computation to obtain it.

Status

Fixed

Gas 02 Raid Latch Fields `raid_lock_started_at`, `raid_lock_status`, `outgoing_raid_started_at` Are Redundant

Location

raid.rs

Description

Three fields on the `Bank` struct related to raid latch timing and status are written during `initiate_raid` and cleared during `settle_raid`/`cleanup_raid_attempt`, but are never read by any on-chain validation function:

- `target.raid_lock_started_at`
- `target.raid_lock_status`
- `attacker.outgoing_raid_started_at`

The lock/latch existence checks (`in_flight_raid_attempt.is_some()`, `outgoing_raid_attempt.is_some()`) use only the `Option<Pubkey>` fields, not the timing or status fields. These fields appear to be intended for off-chain monitoring or future use but currently waste compute on writes that no on-chain logic depends on.

Recommendation

Either:

- Use the fields in on-chain validation (e.g., check `raid_lock_status` to determine latch state), or
- Remove them to save compute units and account space, or
- Document that they are emitted only for off-chain event indexing.

Status

Acknowledged

Gas 03 Unnecessary `mut` On Signer Accounts Across Multiple Instructions

Location

initialize.rs — `BootstrapLiquidityMint` :: `admin`

rewards.rs — `ClaimFarmingRewards` :: `owner`, `ClaimRaidRewards` :: `owner`, `ClaimReferralRewards` :: `owner`

crew.rs — `BatchUpdateCrewAccounts` :: `owner`

fee_vault.rs — `SweepFeeVaultShard` :: `sweeper`, `WithdrawCashTransferFeesFromAccounts` :: `sweeper`, `WithdrawCashTransferFeesFromMint` :: `sweeper`

Description

Four signer fields across the codebase are declared with `mut` or `#[account(mut)]` but are never written to in any handler. Marking a `Signer` as mutable signals to the Solana runtime that the account may be modified, forcing the runtime to take a write lock. This inflates compute unit consumption on every invocation.

The specific occurrences:

1. `initialize.rs` — `admin: Signer<'info>` in `BootstrapLiquidityMint` is marked `#[account(mut)]`. The handler only reads `.key()` for event emission and uses the account as a signer in a CPI call. The actual CPI authority is the `mint_authority` PDA (signing via seeds), not the admin. No lamports are transferred from or to the admin.
2. `rewards.rs` — `owner: Signer<'info>` in all three claim structs (`ClaimFarmingRewards`, `ClaimRaidRewards`, `ClaimReferralRewards`) is marked `#[account(mut)]`. In every handler, the owner is only read via `.key()` for authentication and event emission. No lamports are deducted from or added to the owner's system account.
3. `crew.rs` — `owner: Signer<'info>` in `BatchUpdateCrewAccounts` is declared with `mut`. The field is only read via `.key()` and passed by value as `ctx.accounts.bank.owner`. It is never assigned or written to.
4. `fee_vault.rs` — `sweeper: Signer<'info>` in all three fee-vault account structs is marked `#[account(mut)]`. In each of the three handlers:

- `sweep_fee_vault_shard_handler`
- `withdraw_cash_transfer_fees_from_accounts_handler`
- `withdraw_cash_transfer_fees_from_mint_handler`

the sweeper is only read via `.key()`.

Recommendation

Remove the `#[account(mut)]` attribute (or the `mut` keyword) from each declaration:

```
// initialize.rs
pub admin: Signer<'info>,

// rewards.rs — all three claim structs
pub owner: Signer<'info>,

// crew.rs
pub owner: Signer<'info>,

// fee_vault.rs — all three fee-vault structs
pub sweeper: Signer<'info>,
```

This changes the runtime lock from write to read, eliminating the unnecessary compute-unit overhead.

Status

Fixed